

Projeto: Poker Analytics Engine (PAE)

Documento de Arquitetura, Modelagem e Rotas de Desenvolvimento

Resumo Executivo: Este documento detalha a arquitetura para um sistema de análise de probabilidades e riscos em jogos de Texas Hold'em. O sistema capta o estado do jogo via visão computacional, calcula *equity* em tempo real e fornece decisões baseadas em Valor Esperado (EV).

1. Decisão de Interface: Terminal vs. Overlay Gráfico

Para a primeira versão (MVP), a recomendação absoluta é manter a interface operando diretamente no terminal (`htheoenderson@DESKTOP-END`) e evitar a criação imediata de um overlay gráfico na tela do jogo. Os motivos técnicos para essa abordagem são:

- Velocidade de Desenvolvimento:** Focar na lógica matemática e na leitura de tela sem o gargalo de gerenciar *threads* complexas de interfaces gráficas (como PyQt ou Tkinter).
- Furtividade (Anti-Cheat):** Muitos clientes de poker rastreiam processos que desenham janelas sobrepostas à área do jogo (injeção de overlay). Um script em Python rodando em um terminal e apenas capturando a tela (read-only) é passivo e praticamente invisível.
- Uso de Recursos:** A máquina pode reservar o máximo de processamento e RAM (otimizando o uso dos 12GB disponíveis) para rodar as simulações pesadas de Monte Carlo em vez de renderizar UI.

2. Arquitetura do Sistema e Stack Tecnológico

O sistema será construído de forma modular para separar a leitura de imagem do processamento matemático.

Módulo	Responsabilidade	Tecnologia Recomendada
Visão Computacional	Captura de tela, extração de ROI (Region of Interest) para cartas e OCR numérico (pote/apostas).	Python + OpenCV + Tesseract OCR
Máquina de Estados	Controle de transição do jogo (Pre-Flop → Flop → Turn → River).	Python (Orientação a Objetos)
Motor Matemático	Cálculo de combinações de mãos e simulações de cenários futuros.	C / C++ integrado ao Python via <i>ctypes</i> ou extensões Python.

3. Modelagem: Autômatos e Teoria de Linguagens

Como o Texas Hold'em possui regras sequenciais estritas, o núcleo lógico do sistema deve ser construído sobre o conceito de uma Máquina de Estados Finitos (FSM). Isso previne bugs comuns de leitura (ex: o sistema achar que está no River enquanto a imagem do jogo está travada no Flop).

As transições de estado dependem dos *inputs* lidos pela visão computacional:

- Estado `WAITING_HAND` → Transição: (Detecta cartas do "Hero") → Estado `PRE_FLOP`
- Estado `PRE_FLOP` → Transição: (Detecta 3 cartas comunitárias) → Estado `FLOP`
- Estado `FLOP` → Transição: (Detecta 1 carta comunitária adicional) → Estado `TURN`

A definição formal dessa gramática do jogo garantirá que o bot não tome decisões com dados dessincronizados.

4. O Motor de Probabilidades (Python + C)

Em cada estado da mão, precisamos calcular a probabilidade de vitória (*Equity*). A forma mais precisa de fazer isso contra um range de mãos desconhecidas dos adversários é usando simulações de Monte Carlo.

Otimização de Performance: Em vez de escrever o laço principal de simulação (milhões de iterações por segundo) em Python puro, a estratégia recomendada é construir o simulador base em C puro. Você compila esse código em uma biblioteca dinâmica (via GCC ou outro compilador) e consome os dados no Python usando a biblioteca `ctypes`. Assim, a interface de linha de comando no terminal fica rápida e fluida.

5. Análise de Risco (Valor Esperado - EV)

Toda decisão (Call, Raise, Fold) será filtrada através da equação de EV (Valor Esperado). O sistema processará a equação e imprimirá o resultado final de rentabilidade:

$$EV = (P_{win} \times \$V_{\{pot\}}) - (P_{lose} \times \$V_{\{bet\}})$$

- P_{win} = Probabilidade de ganhar a mão (calculada em C).
- $\$V_{\{pot\}}$ = Lucro potencial (tamanho total do pote lido via OCR).
- P_{lose} = Probabilidade de perder ($1 - P_{win}$).
- $\$V_{\{bet\}}$ = Quantia que deve ser paga (Risco, lido via OCR).

6. Roadmap de Desenvolvimento

1. **Fase 1 (Sincronização):** Iniciar repositório Git, tirar dezenas de *screenshots* do Replay Poker. Desenvolver o *Template Matching* com OpenCV para detectar os naipes e valores de forma isolada, sem pressa.
2. **Fase 2 (OCR e Estados):** Calibrar o Tesseract para ler perfeitamente o tamanho do pote e os stacks. Estruturar a classe principal da Máquina de Estados.
3. **Fase 3 (Core Matemático):** Escrever o avaliador de mãos, conectar a lógica em C com Python, validar contra calculadoras de poker públicas para testar a acurácia.
4. **Fase 4 (Integração):** Fazer o loop rodar em tempo real no terminal durante o jogo, exibindo a recomendação de aposta em tempo real.